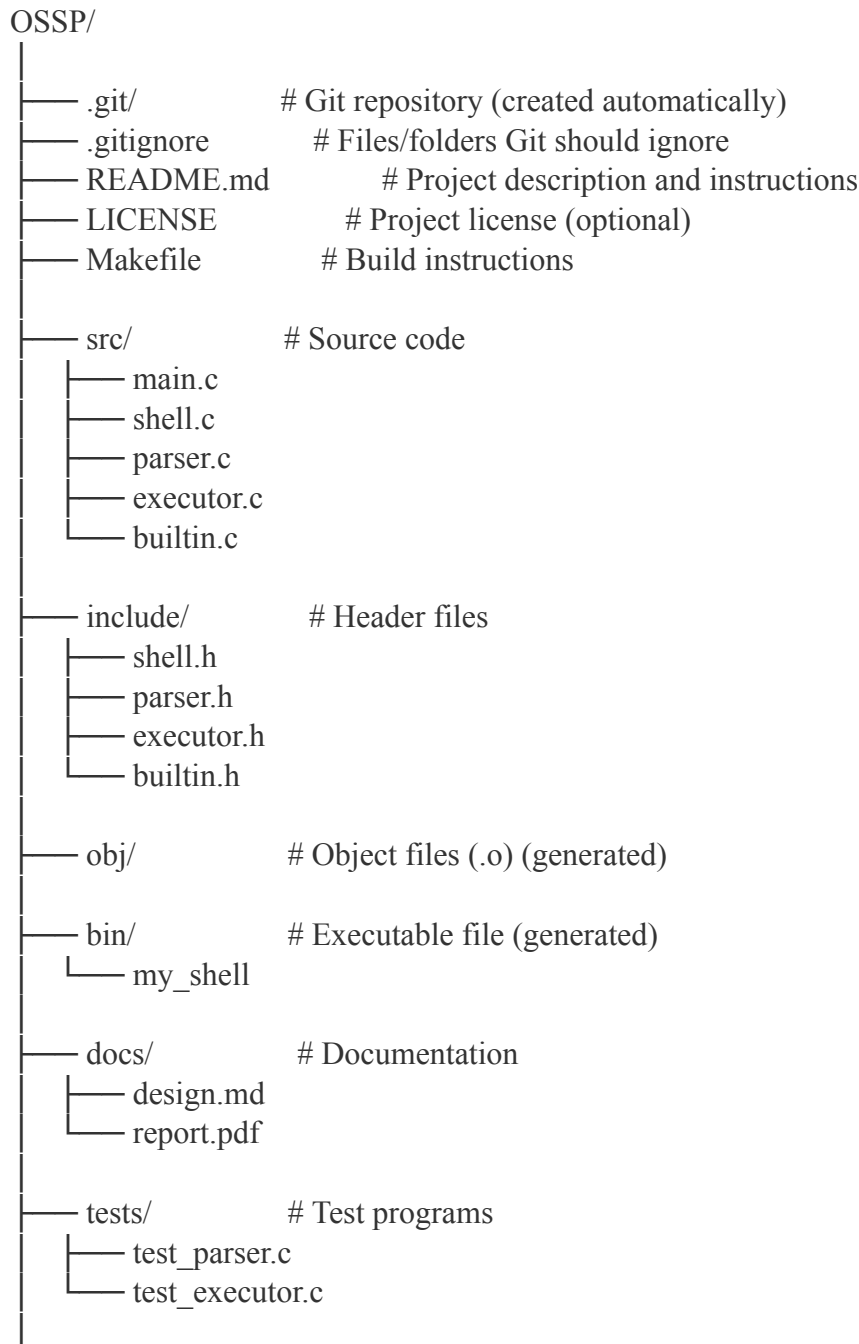


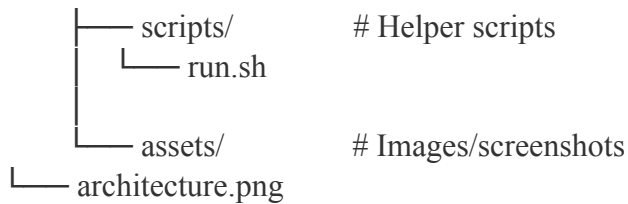
## SKILL WEEK-1

### Session-1:

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

**Task -1:** Create Project Structure as follows:





Note: For creating this structure use mkdir and touch commands.

## OUTPUT and OBSERVATION :

```

ananya@Ananya:~/OSSP$ tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── fork_example
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── practical2
├── practical2.c
├── scripts
│   └── run.sh
├── skill2
├── skill2.c
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

```

The OSSP project directory structure was successfully created in WSL Ubuntu using mkdir and touch commands. The required source, header, documentation, test, script, object, binary, and asset directories were created successfully. The project structure was verified using the tree command.

**2.To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.**

**Task-1:** Using a manual command understand the fork() and exec() system calls

## OUTPUT and OBSERVATION :

```

ananya@Ananya:~/OSSP$ bash
ananya@Ananya:~/OSSP$ ps
  PID TTY          TIME CMD
  447 pts/0        00:00:00 bash
  606 pts/0        00:00:00 bash
  613 pts/0        00:00:00 ps
ananya@Ananya:~/OSSP$ cd /mnt/d/OSSP_2520080069/Skill/Week1
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week1$ bash
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week1$ ps
  PID TTY          TIME CMD
  447 pts/0        00:00:00 bash
  606 pts/0        00:00:00 bash
  624 pts/0        00:00:00 bash
  631 pts/0        00:00:00 ps
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week1$ exec ls
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week1$ █

```

The ps command displayed multiple Bash processes, demonstrating process creation and process abstraction. The exec ls command replaced the current Bash process with the ls program, demonstrating that exec() replaces the existing process rather than creating a new one.

**Task-2:** Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

**Example:**

```

#include <stdio.h>

#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork(); // Create a new process

    if (pid < 0) {
        printf("Fork failed!\n");
    }

    else if (pid == 0) {
        // Child Process

        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
}

```

```

    }
    else {
        // Parent Process

        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}

```

### OUTPUT and OBSERVATION :

```

ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week1$ nano fork_exec.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week1$ gcc fork_exec.c -o fork_exec
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week1$ ./fork_exec
This is the Parent Process.
Parent PID = 652
Child PID = 653
This is the Child Process.
Child PID = 653
Child is executing 'ls'...
fork_exec fork_exec.c
Child process completed.
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week1$ █

```

The program successfully created a child process using `fork()`. The parent and child processes were identified using their PIDs. The child used `exec()` to execute the `ls` command, while the parent waited for the child to complete using `wait()`. This demonstrated basic parent-child process creation and execution in Linux.

### Session-2:

**Task 1:** To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ nano task1.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ gcc task1.c -o task1
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./task1
Enter a command (type 'exit' to quit): ls
You entered: ls
Enter a command (type 'exit' to quit): pwd
You entered: pwd
Enter a command (type 'exit' to quit): exit
Exiting program...
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ |
```

The main interactive shell loop was successfully created and executed. The program displayed a prompt and continuously accepted commands from the user. The entered commands, such as `ls` and `pwd`, were successfully read and displayed. The `exit` command was recognized as an exit condition, after which the program terminated and returned to the WSL terminal.

**Task 2:** Using the following system calls, copy the content from `File1.txt` to `File2.txt`.

1. `Open()`
2. `Read()`
3. `Write()`
4. `Close()`

```
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ nano task2.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ gcc task2.c -o task2
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./task2
File copied successfully!
```

The program successfully copied the contents of `File1.txt` to `File2.txt` using the Linux system calls `open()`, `read()`, `write()`, and `close()`. The file was opened and its contents were read into a buffer, then written to the destination file. The program displayed “File copied successfully!”, confirming that the file-copy operation was completed successfully.

**Task 3:** To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
ananya@Ananya:~$ pwd
/home/ananya
ananya@Ananya:~$ cd /mnt/d/OSSP_2520080069/Skill/Week3
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ls
File1.txt File2.txt task1 task1.c task2 task2.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ nano task3.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ gcc task3.c -o task3
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./task3
Simple Command Shell
Type a command and press Enter.
Type 'exit' to quit.

> hello
You entered: hello
> hello
You entered: hello
> exit
You entered: exit

Program terminated.
```

The simple command shell was successfully implemented to capture and process keyboard input. The program accepted multiple commands from the user and displayed the entered input correctly. The input was processed after pressing the Enter key, demonstrating basic input-buffer management. The exit command was also handled correctly, causing the shell program to terminate and display the termination message.